

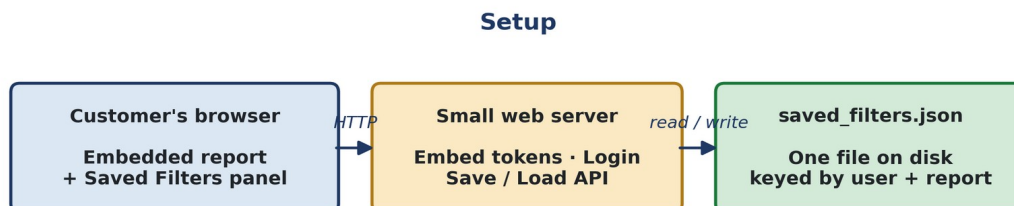
Power BI Saved Slicers — Handover

How customers save and restore slicer selections on the embedded report.

What it does

Embedded Power BI report customers can save their slicer selections under a name, see their list of saved views on return, and apply any of them in one click. Saves survive logins and are isolated per customer and per report.

Setup at a glance



Customer signs in (name/email). Page receives the Power BI embed token and uses the JS client to capture and restore slicers.

A single HTML page embeds the report and shows the Saved Filters panel. A small server issues Power BI embed tokens, remembers the customer's name or email (POC-level identity), and reads and writes one JSON file. Every save is keyed by the customer + the report ID.

Key notes — why the design looks the way it does

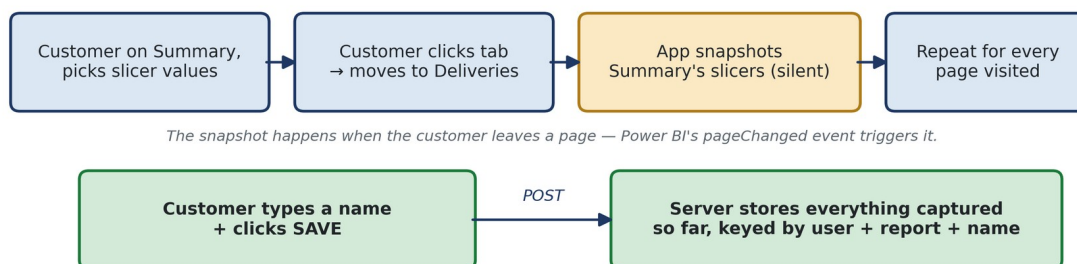
- Our embedded report is a **master dashboard with multiple pages** (Summary, Deliveries, Aircraft Utilization, Airline Finance, BS and PL, Aircraft Specifics, ...). Each page has its own slicers, so a meaningful "saved view" spans more than one page.
- Power BI loads visuals *lazily* — `page.getVisuals()` only returns visuals on the page that is currently in front of the customer. So a saved view that touches 6 pages can't be read or restored in one go without flipping pages.
- `report.on('pageChanged', ...)` is what makes the multi-page experience smooth: every time the customer changes page, we get a callback. On that callback we snapshot the slicers they're leaving (for Save) and push any waiting saved-view slicers for the new page (for Apply). No forced page tour, no flicker.
- `report.on('loaded', ...)` fires once when the report is ready — we use it to seed the slicer cache for the starting page and to auto-apply the customer's most-recent saved view on a fresh login.

Power BI client functions used

- `report.getFilters` / `setFilters` / `updateFilters` — read and write report-level filters (the right-hand Filters pane).
- `report.getPages` and `page.getVisuals` — discover pages and visuals on the active page.
- `visual.getSlicerState` / `setSlicerState` — read and write a single slicer's selection. Empty `{ filters: [] }` wipes the slicer.
- `report.on('pageChanged' | 'loaded' | 'rendered' | 'error', ...)` — lifecycle and navigation events. `pageChanged` is the workhorse.

What happens when you click Save

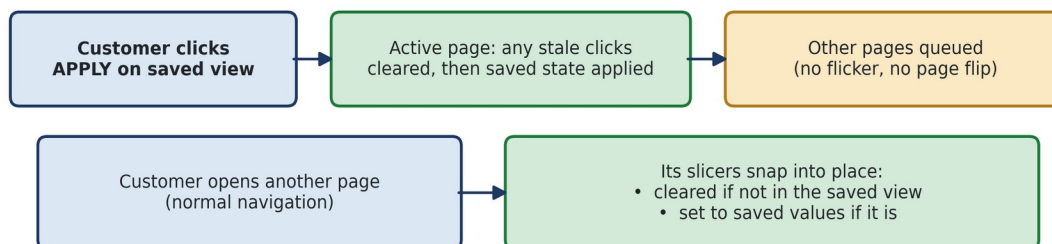
SAVE — captured silently as the customer navigates



- On every `pageChanged`, the app silently snapshots the slicers on the page being left, via `getSlicerState`. No flicker.
- On click, the active page is refreshed into the cache and the whole bundle is POSTed and stored under the customer's name.

What happens when you click Apply

APPLY — current page now; other pages on navigation



- Active page: any slicer with a stale selection is cleared (empty `setSlicerState`), then the saved state is pushed.
- Other pages are queued. On the next `pageChanged`, the same clear-then-set runs, so switching between saved views always lands on a clean result.

Production notes

- Identity: swap the name/email box for real customer login (SSO, JWT, magic link). Nothing else needs to change on the server side.
- Storage: the JSON file is fine for a POC with a few hundred users. At 10k+ customers the file grows to hundreds of MB and every Save rewrites the whole thing — move to a database (one row per saved view, keyed on user + report + name) or your Lakehouse.
- Only pages the customer actually visits in a session contribute to a Save; custom third-party slicer visuals may not be readable.